

[3조 최종 보고서]

AI 기반 사용자 인증 및 보안 시스템 개발

20231900 김희서: 백엔드 아키텍처 및 데이터베이스

20206830 유민성: 키스트로크, RBA 연구 및 보안 로직 개발

20232110 최윤서: 프론트엔드 개발 및 UI/UX 설계

****설계 프로젝트 저장소(Github): <https://github.com/hs03290811/capstone2>**

1. 서론

1.1 프로젝트 배경 및 필요성

최근 유출된 사용자 계정 정보를 다른 서비스에 무작위로 대입하여 계정을 탈취하는 크리덴셜 스테핑(Credential Stuffing) 공격이 빈번하게 발생하고 있다. 국내에서도 GS리테일(9만 명 유출), 티머니, 대성마이맥 등 주요 기업들이 해당 공격으로 인해 큰 피해를 입었다. 이러한 계정 탈취는 단순한 로그인 성공에 그치지 않고, 탈취된 권한을 이용한 2차 위협으로 확장될 가능성이 크다.

대상 기업	시기	피해 규모 및 내용
GS리테일	'24.06 - '25.02	약 158만 건 공격 시도, 9만 명의 개인정보 유출
티머니	'25.04	5만여 명 계정 로그인 성공, T마일리지 1,400만 원 탈취
대성마이맥	'24.01	크리덴셜 스테핑 및 XSS 공격으로 9만 5천여 명 정보 탈취

사업자명	위반 내용	위반 조항	시정조치(안)
(주)디지털대성	• 안전조치의무 위반 • 유출 통지·신고 위반	보호법 \$29, \$34①	• 과징금 6억 1,300만 원 • 과태료 330만 원 • 공표명령*
< 개인정보 유출 경위 >			
사업자명	위반 내용	위반 조항	시정조치(안)
(주)하이컨시	• 안전조치의무 위반 • 유출 통지·신고 특례 위반	舊 보호법 \$29, \$39의4①	• 과징금 2억 8,000만 원 • 과태료 1,020만 원 • 공표

1.2 기존 인증 방식의 한계

현재 가장 대중적인 대응책인 다요소 인증(MFA)은 계정 탈취 공격을 약 98.56%까지 차단하는 높은 보안성을 보이지만, 사용자가 매번 추가 인증을 수행해야 하는 번거로움이 있어 실제 사용률이 낮고 기피되는 경향이 있다. 반면 단일 인증(SFA)은 편리하지만 크리덴셜 스테핑과 같은 공격에 매우 취약하다.

1.3 연구의 목적 및 목표

본 연구는 비밀번호가 이미 유출된 최악의 상황을 가정한다. 단순히 유출을 막는 단계를 넘어, 비밀번호가 노출되더라도 실제 사용자를 안전하게 식별할 수 있는 시스템 구축을 목표로 한다. 이를 위해 사용자의 타건 리듬을 분석하는 '키스트로크 다이내믹스'와 접속 환경을 분석하는 '위험 기반 인증(RBA)' 기술을 결합한다. 최종적으로 보안 수준은 극대화하면서 사용자의 번거로움은 최소화하는 "구조는 MFA, 경험은 SFA"인 시스템을 구현한다.

2. 키스트로크 다이내믹스 기반 인증 고도화

2.1 데이터셋 도입 및 특징 추출

초기 실험에서는 7자리 고정 비밀번호 데이터를 자체 수집했으나, 과적합 문제로 인해 유의미한 결과를 얻지 못했다. 이를 해결하기 위해 51명의 타건 데이터가 담긴 **CMU 벤치마크 데이터셋**을 도입한다. 해당 데이터셋에서 핵심 속성인 **Hold Time(H)**, **DD(Press-to-Press)**, **UD(Release-to-Press)** 세 가지를 추출하여 분석에 활용한다.

속성 구분	상세 정의
H Hold Time	하나의 특정 키를 누르고 있는 시간 (Keydown to Keyup)

속성 구분	상세 정의
DD Keydown-Keydown	앞의 키를 누른 시점부터 다음 키를 누른 시점까지의 간격
UD Keyup-Keydown	앞의 키를 뗀 시점부터 다음 키를 누른 시점까지의 물리적 공백 시간

2.2 모델 최적화 및 알고리즘 선정

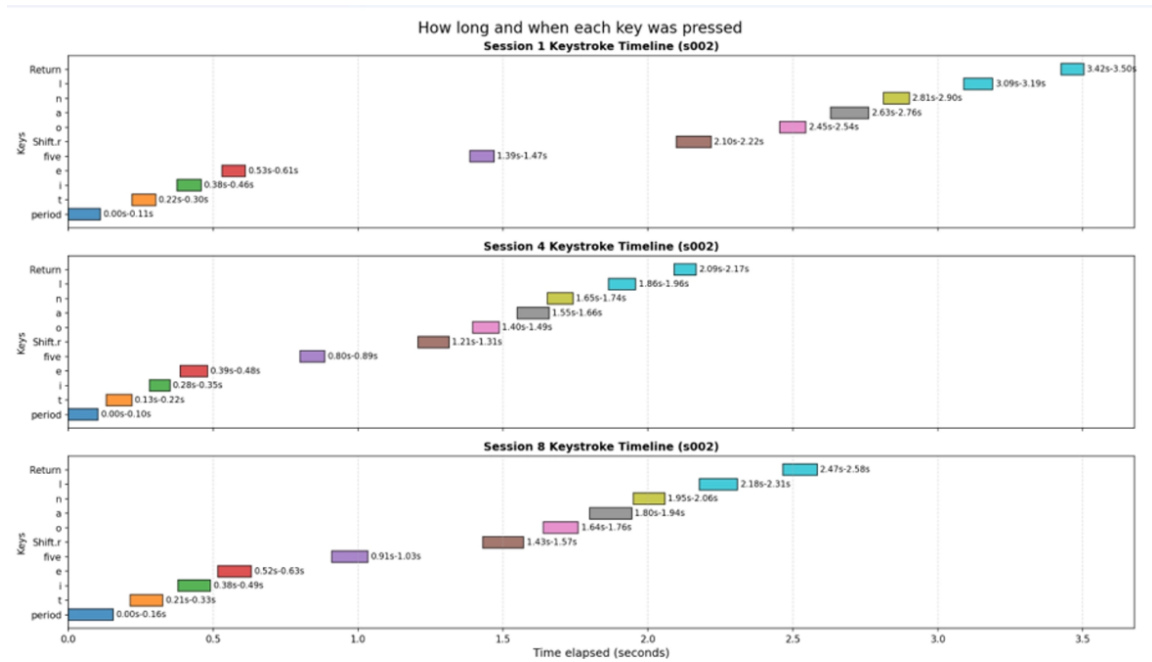
Standard Scaler와 Robust Scaler를 활용한 전처리를 진행하고, 오인식률(FAR)과 오거부율(FRR)이 일치하는 지점인 **EER**을 최종 성능 지표로 설정한다. 실험 결과, 연산 복잡도가 낮으면서도 최상위권의 보안 성능(EER 8.58%)을 보인 맨해튼 거리(Manhattan Distance)를 최종 알고리즘으로 선정한다. 이는 소량의 데이터셋만으로도 사용 가능한 경량성을 갖추고 있다.

실험 조건	EER
맨해튼 거리(Robust scaler, LOSO-CV)	10.52%
맨해튼 거리(Standard scaler, LOSO-CV)	8.58%
맨해튼 거리(Standard scaler, LOO-CV)	8.51%
Isolation Forest	8.74%
One-class svm(K-S 검정 + KNN 전처리, LOSO-CV)	10.60%

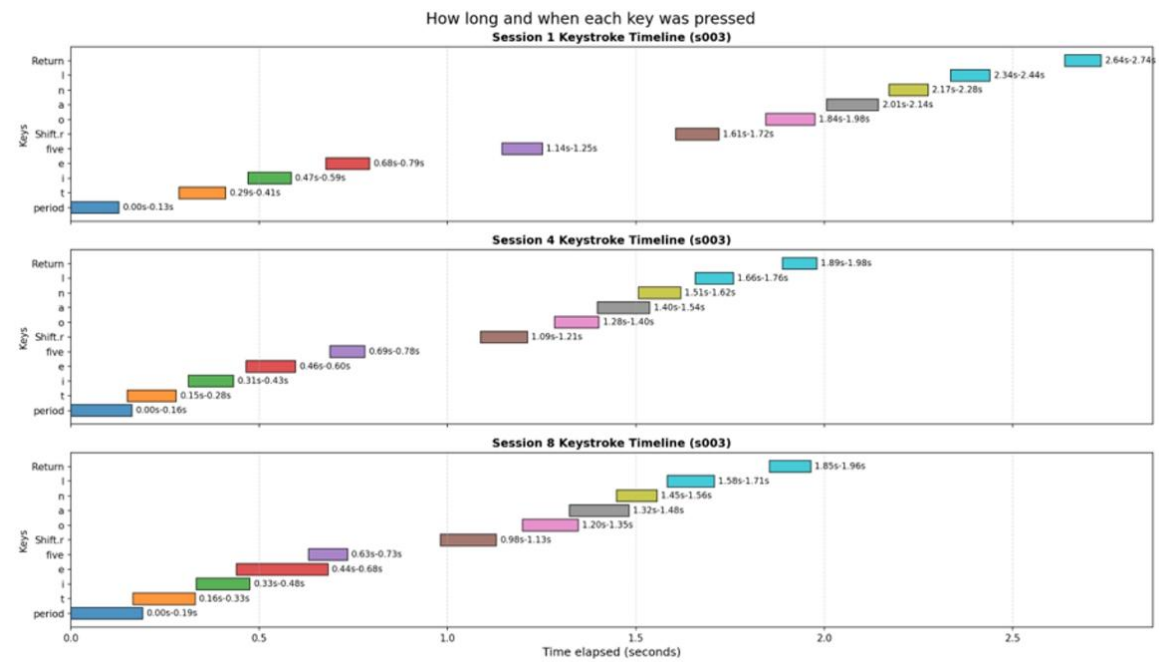
2.3 시간적 흐름 반영 및 정밀도 향상

사용자 타건 패턴 시각화 결과, 세션이 반복될수록 입력 시간이 짧아지는 시계열적 변화가 관찰된다.

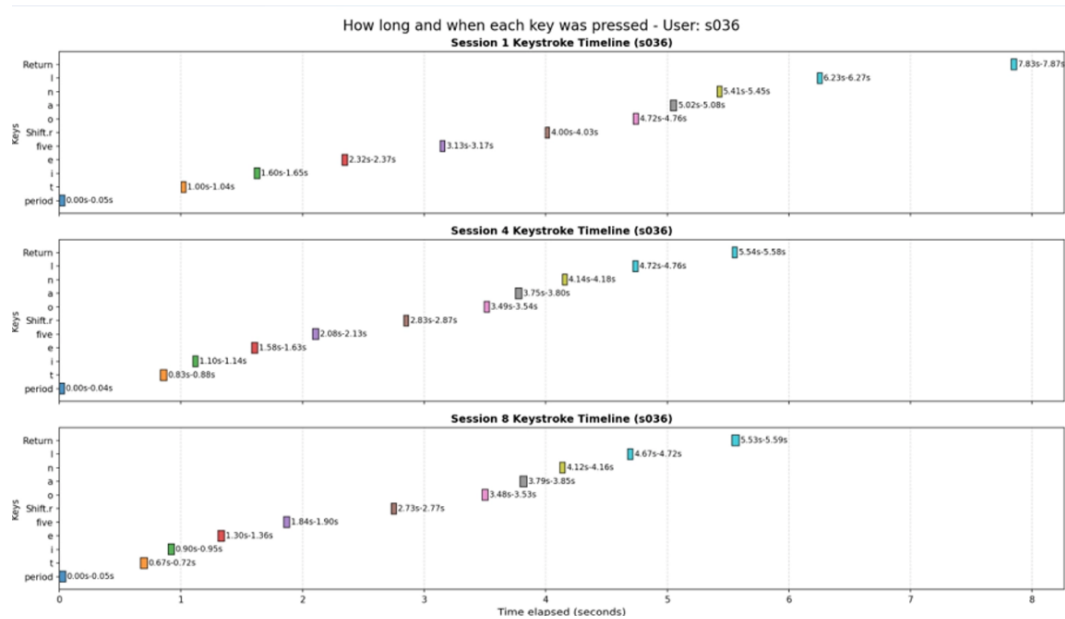
Subject'002' 세션 별 평균 입력



Subject'003' 세션 별 평균 입력



Subject'036' 세션 별 평균 입력



이를 반영하기 위해 **슬라이딩 윈도우(Sliding Window)** 방식을 적용하며, 실험을 통해 최적의 윈도우 크기를 '3'으로 결정하여 EER을 7.65%까지 개선한다. 또한 비밀번호 길이에 따른 왜곡을 방지하는 '피쳐 개수 정규화'와 사용자의 기복을 고려한 '**동적 임계값(k=3)**' 기법을 적용하여 오거부율(FRR)을 3.13%까지 낮춘다.

윈도우 크기(Window Size)별 전체 평균 EER

실험 조건	EER
Window 크기 1	9.24%
Window 크기 2	7.79%
Window 크기 3	7.65%
Window 크기 4	7.63%
Window 크기 5	7.70%

윈도우 크기 3 고정 시 k값 변화 실험 결과

파라미터 조건	FRR (오거부)	FAR (오인식)	비고
k = 2 적용	6.49%	11.67%	-
k = 3 적용	3.13%	22.32%	최종 선정
k = 4 적용	1.68%	32.63%	-

3. 위험 기반 인증(RBA) 시스템 구현

3.1 데이터 모델링 및 실환경 데이터셋 도입

초기 가상 시나리오 데이터는 모든 평가지표에서 1.0(100%)을 기록하며 변별력 확보에 실패한다. 이에 노르웨이 온라인 서비스의 **대규모 실환경 데이터셋**을 새롭게 도입하고, 결측치 제거 등 전처리를 거쳐 실제 위험 탐지에 핵심적인 속성들을 추출한다.

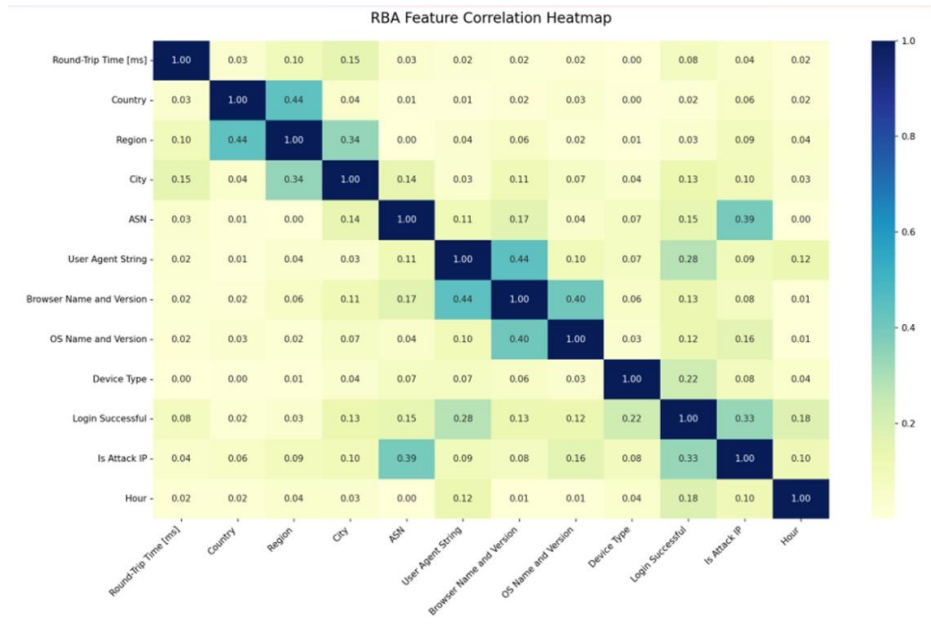
3.2 모델 성능 비교 및 최종 모델 채택

KNN과 랜덤 포레스트(Random Forest)를 비교 평가한 결과, 모든 지표에서 우수하고 특히 위험 탐지율인 **재현율(Recall)**에서 **99.02%**라는 압도적인 성능을 보인 **랜덤 포레스트**를 최종 모델로 채택한다. 랜덤 포레스트는 범주형 및 수치형 데이터를 효과적으로 처리하며 보안 모델의 설명 가능성을 확보할 수 있다는 장점이 있다.

모델명	Accuracy	Precision	Recall	F1-Score
KNN 모델	0.9810	0.9619	0.8806	0.9152
Random Forest	0.9975	0.9892	0.9902	0.9897

3.3 속성 중요도 분석 및 위험 스코어링 설계

스피어만 상관관계 분석을 통해 특정 속성 배제에 따른 성능 변화를 실험한다. 지리 정보(City, Region)는 중복성이 높으나, **'User Agent String'** 속성은 배제 시 재현율이 급감하여 미탐 위험이 증가함을 확인한다.

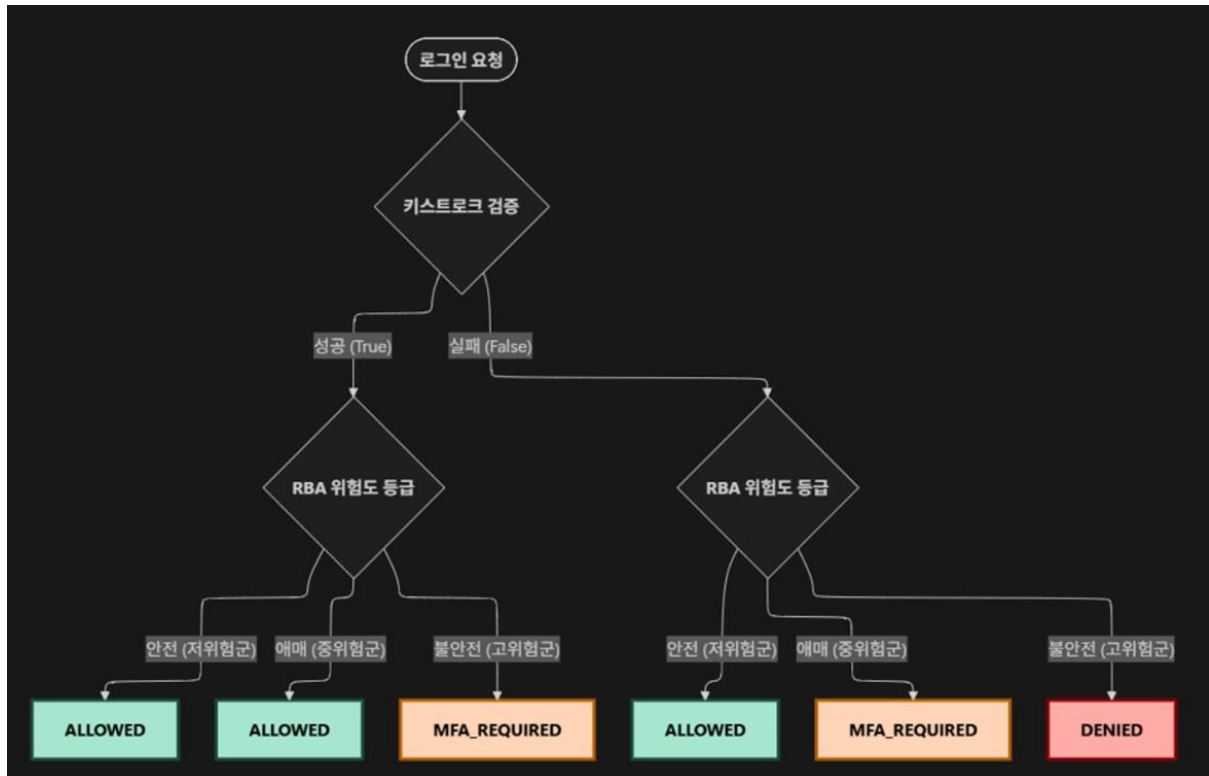


이러한 분석 결과를 바탕으로 **3단계 인증 체계**를 구축한다. 접속 환경 신뢰도가 75점 이상인 '**안전(Safe)**' 등급은 추가 인증 없이 로그인을 승인하고, '중간'이나 '위험' 등급은 단계별 추가 검증을 요구하도록 설계한다.

4. 시스템 아키텍처 및 인증 로직

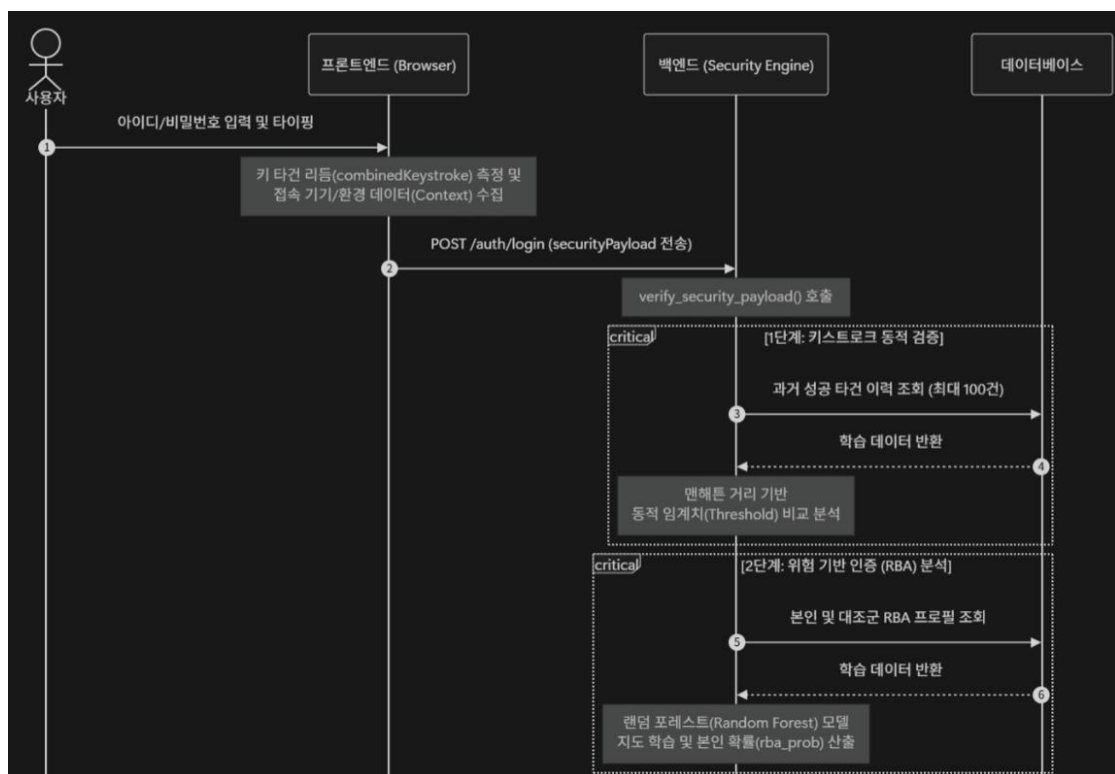
4.1 통합 인증 로직 설계

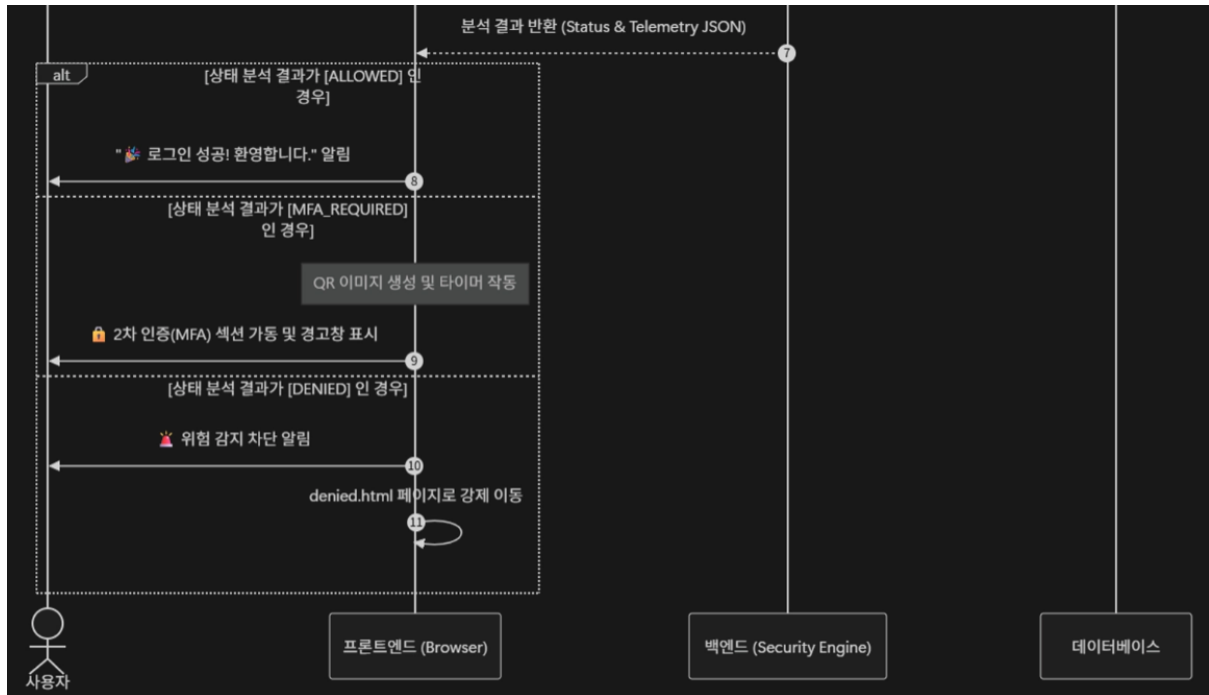
본 시스템은 신뢰도가 높은 RBA를 주된 판단 근거로 삼고 키스트로크 다이내믹스를 보조 지표로 활용한다. 플로우 차트에 따르면 키스트로크 검증에 실패하더라도 RBA 등급이 안전할 경우 최종 로그인을 승인하여 사용자 편의성을 보장한다.



4.2 데이터 교환 프로세스

로그인 시 프론트엔드는 타건 데이터와 환경 정보를 수집하여 서버로 전송한다. 서버는 이를 기존 사용자의 프로필 및 과거 로그와 대조 분석하여 즉시 승인 혹은 2차 인증 요구 여부를 결정한다.





5. 시스템 구현 상세

5.1 프론트엔드 구현

5.1.1 데이터 수집 프로세스

본 시스템의 프론트엔드는 엔드포인트(Endpoint) 단에서 사용자의 고유한 행위 생체 정보(Behavioral Biometrics)를 밀리초(ms) 단위로 정밀 계측하고, 클라이언트의 다차원 환경 인자를 동적으로 수집하여 인공지능 분석을 위한 실시간 데이터 파이프라인을 구축합니다.

(1) Keystroke Dynamics 계측 및 데이터 정제

사용자가 패스워드 입력 필드에 텍스트를 입력하는 과정에서 비동기 이벤트 리스너를 통해 keydown 및 keyup 이벤트를 수집하고, performance.now() 기반의 고정밀 타임스탬프를 기록한다. 수집된 이벤트 정보는 후속 연산을 통해 Keystroke Dynamics 분석에 활용되는 특징 벡터로 변환된다.

본 시스템은 다음과 같은 세 가지 핵심 특징값을 추출한다.

- **Hold Time (H)** : 특정 키를 누른 시점부터 떼어낸 시점까지의 유지 시간 (keydown → keyup)
- **Down-Down Time (DD)** : 이전 키를 누른 시점부터 다음 키를 누른 시점까지의

시간 간격 (keydown → keydown)

- **Up-Down Time (UD)** : 이전 키를 댄 시점부터 다음 키를 누를 때까지의 시간 간격 (keyup → keydown)

추출된 특징값은 사용자의 타이핑 패턴을 정량적으로 표현하는 벡터 형태로 구성되며, 이후 사용자 인증 과정에서 기준 프로필과의 유사도 분석에 활용된다.

(2) RBA 기반 환경 정보 수집

본 시스템은 키스트로크 데이터 분석과 함께 위험 기반 인증(RBA, Risk-Based Authentication)에 활용되는 사용자 환경 정보를 수집한다. 수집된 환경 정보는 로그인 시 사용자의 접속 환경을 식별하고 위험도를 평가하는 데 활용한다.

- **실시간 네트워크 지연 시간 측정**: 브라우저의 PerformanceNavigationTiming API를 활용하여 페이지 요청 시점부터 응답 완료 시점까지의 시간을 계산하고, 네트워크 응답 속도(RTT, Round Trip Time)를 측정한다.
- **위치 및 네트워크 정보 수집**: 외부 위치 정보 서비스(<https://ipapi.co/json/>)를 활용하여 사용자의 공인 IP 주소, 국가(Country), 지역(Region), 도시(City), 통신사 및 네트워크 정보(ASN)를 수집한다.
- **디바이스 및 브라우저 정보 수집**: 화면 해상도(window.screen), 브라우저 언어 설정(navigator.language), 운영체제 및 브라우저 식별 정보(navigator.userAgent)를 수집한다.

수집된 환경 정보는 키스트로크 분석 결과와 함께 백엔드로 전달되며, 로그인 시 접속 환경의 신뢰도를 평가하기 위한 RBA 분석 데이터로 활용한다.

백엔드 전송 규격 데이터 패킷: script.js:159

```

▼ Object
  asn: "SK Broadband Co Ltd"
  browser_name_version: "Chrome/149.0.0.0"
  city: "Dongjak-gu"
  country: "South Korea"
  device_type: "Desktop"
  ip_address: "219.255.207.61"
  ▶ keystroke: (16) [203, 133, -70, 141, 247, 106, 161, 124, -37, 137, 118, -
    language: "ko-KR"
    os_name_version: "Windows 10/11"
    password: "*****"
    region: "Seoul"
    resolution: "1707x1067"
    rtt: 8
    user_agent_string: "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit
    username: "RBA 안전"
  ▶ [[Prototype]]: Object

```

(3) 데이터 정규화 및 백엔드 DTO 인터페이스 통합

프론트엔드에서 수집한 환경 정보와 디바이스 식별 정보는 전송 전 정규화 과정을 수행한다. 수집된 데이터는 백엔드 API 서버에서 요구하는 DTO(Data Transfer Object) 규격에 맞도록 구조화하며, 환경 정보와 키스트로크 특징 벡터를 하나의 JSON 객체로 통합한다.

통합된 데이터는 securityPayload 객체 형태로 구성하며, 비동기 통신을 통해 백엔드 API 서버로 전송한다. 이를 통해 클라이언트 환경 정보와 키스트로크 데이터를 일관된 형식으로 전달하고, 백엔드 분석 모듈에서 활용할 수 있도록 한다.

5.1.2 백엔드 분석 결과 기반 적응형 보안 제어 및 인터페이스 분기

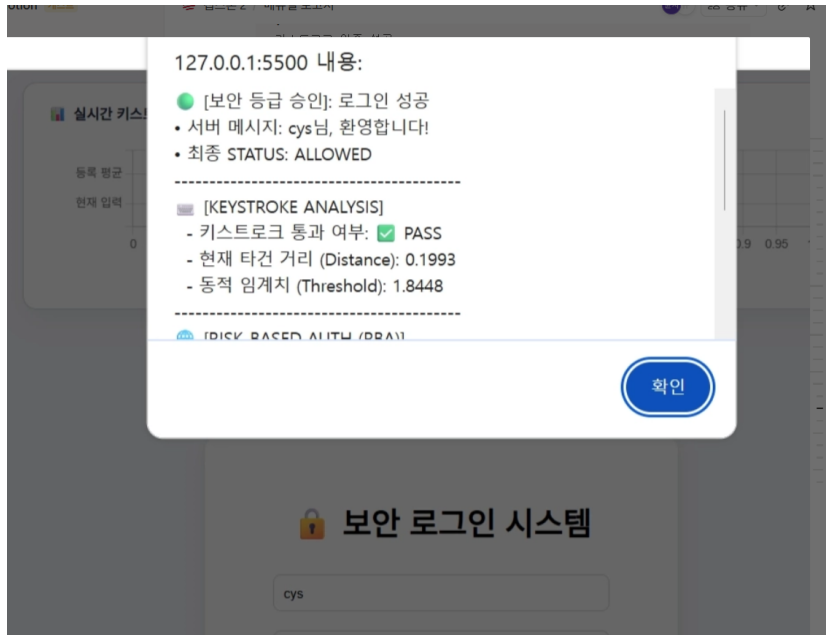
프론트엔드는 로그인 요청 시 수집된 키스트로크 특징 벡터와 환경 정보를 백엔드 API 서버로 전송한다. 백엔드는 키스트로크 분석 결과와 위험 기반 인증(RBA) 분석 결과를 종합하여 최종 인증 상태(Status)를 반환하며, 프론트엔드는 해당 결과에 따라 사용자 인터페이스를 동적으로 제어한다.

본 시스템은 Defense Matrix 기반의 적응형 인증 절차를 적용하며, 분석 결과에 따라 다음과 같이 동작한다.

(1) 로그인 승인 (ALLOWED)

키스트로크 분석과 RBA 분석 결과 모두 정상 범위로 판단된 경우 로그인을 승인한다.

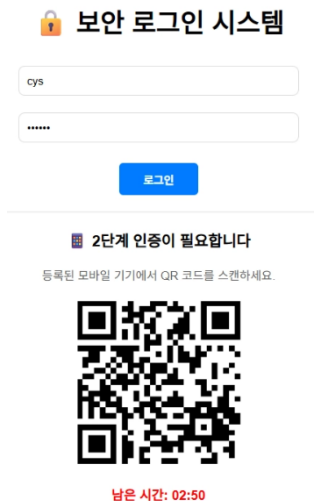
프론트엔드는 로그인 성공 메시지를 출력하고 서비스 접근을 허용한다. 또한 활성화되어 있는 MFA 인터페이스가 존재하는 경우 이를 비활성화하여 정상 세션 환경을 구성한다.



(2) 추가 인증 요구 (MFA_REQUIRED)

키스트로크 분석 또는 RBA 분석 과정에서 추가 검증이 필요하다고 판단된 경우 MFA(Multi-Factor Authentication)를 요구한다.

프론트엔드는 MFA 영역을 활성화하고 QR 코드를 생성하여 사용자 모바일 기기와 연동한다. 사용자는 QR 코드를 통해 추가 인증을 수행하며, 시스템은 180초 제한 시간 내 인증을 완료하도록 카운트다운 타이머를 제공한다.



(3) 접근 차단 (DENIED)

키스트로크 분석과 RBA 분석 결과에서 모두 높은 위험도가 탐지된 경우 접근을 차단한다.

프론트엔드는 위험 경고 메시지를 출력한 후 접근 차단 페이지로 이동시켜 추가적인 서

비스 접근을 제한한다.

또한 인증 결과와 함께 키스트로크 Distance, 동적 Threshold, RBA 위험 등급(Risk Tier), 정상 사용자 확률(Genuine Probability) 등의 분석 정보를 사용자에게 제공한다.

비정상 접근 차단

사용자님의 보안 타이핑 리듬 및 접속 환경이 일치하지 않습니다.

계정 도용 위험이 감지되어 시스템 접근을 즉시 차단합니다. 관리자에게 문의하세요.

[로그인 화면으로 돌아가기](#)

5.1.3 사용자 타이핑 패턴 비교를 위한 실시간 시각화 인터페이스 구현

본 시스템은 사용자의 타이핑 패턴을 직관적으로 확인할 수 있도록 Chart.js 기반의 실시간 시각화 인터페이스를 구현하였다.

로그인 과정에서 수집된 키스트로크 특징 벡터와 회원가입 시 등록된 기준 프로필을 Scatter Chart 형태로 시각화하여 두 패턴 간의 유사도를 비교할 수 있도록 한다.

시각화 과정에서는 키 입력 이벤트를 기반으로 생성된 Hold Time, Down-Down Time 및 Up-Down Time 정보를 이용하여 각 키의 누름(Press) 시점과 땀(Release) 시점을 계산한다.

사용자의 전체 입력 시간은 로그인 시마다 달라질 수 있으므로, 모든 데이터 포인트를 0.0~1.0 범위로 정규화하여 동일한 기준에서 비교한다. 이를 통해 절대적인 입력 속도가 아닌 상대적인 타이핑 리듬의 차이를 분석할 수 있다.

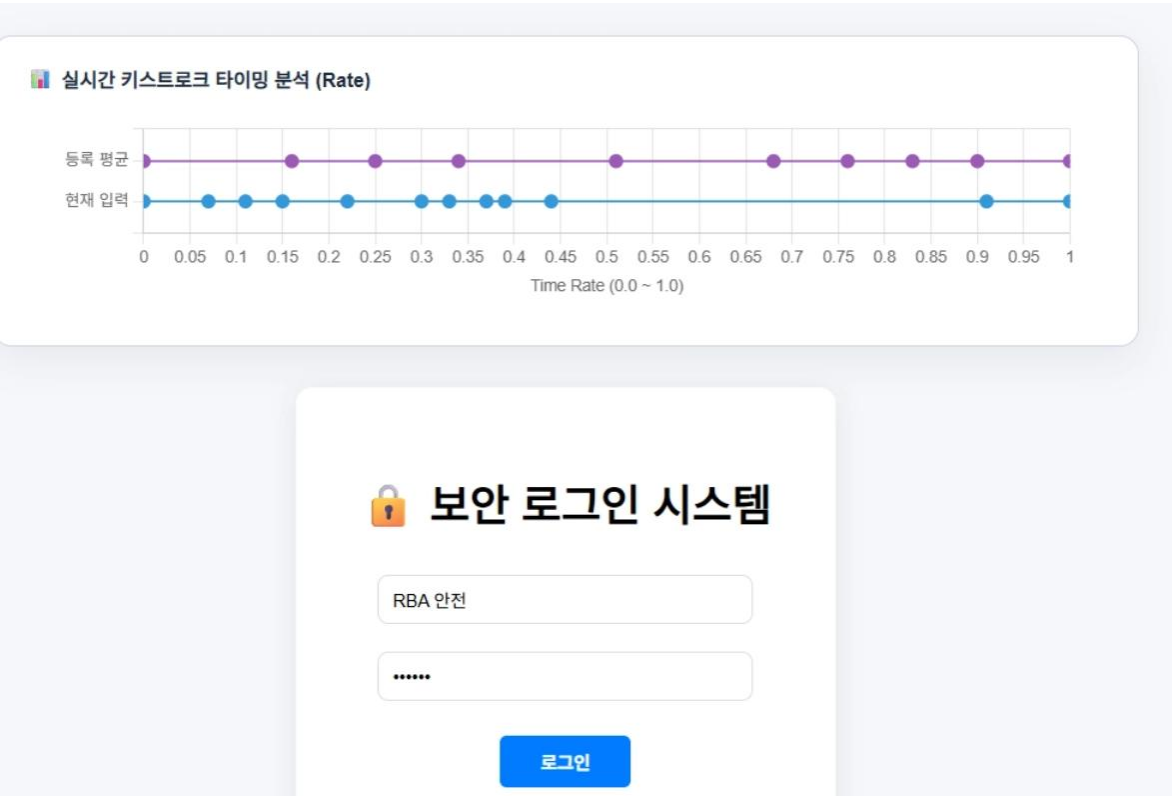
그래프에는 다음 두 개의 데이터셋이 동시에 표시된다.

- 등록 평균 프로필(Registration Profile)
- 현재 로그인 입력 패턴(Current Typing Pattern)

또한 X축 눈금 간격을 0.05 단위로 고정하고 자동 눈금 생략 기능을 비활성화하여 데이터 포인트 위치를 보다 세밀하게 확인할 수 있도록 구성하였다.

최종적으로 프론트엔드는 키스트로크 Distance, 동적 Threshold, 인증 결과(Status)를 함

게 제공하여 사용자가 인증 결과를 직관적으로 확인할 수 있도록 한다.



5.2 백엔드 데이터 관리

Backend Technology Stack

구분	기술
Language	Python 3.11
Framework	FastAPI
ORM	SQLAlchemy
Database	PostgreSQL
Cache	Redis
AI / ML	Scikit-learn
Data Processing	Pandas, NumPy

구분	기술
Authentication	JWT
Container	Docker
Cloud	AWS EC2
Reverse Proxy	Nginx
Version Control	GitHub

5.2.1 백엔드 시스템 개요

본 프로젝트의 백엔드는 사용자 인증, 데이터 저장, 인증 이력 관리, 키스트로크 데이터 관리 및 위험 기반 인증(RBA)을 위한 데이터 파이프라인 구축을 담당한다. 프론트엔드에서 수집한 사용자 입력 데이터와 환경 정보를 수신하여 데이터베이스에 저장하고, 인증 과정에 필요한 데이터를 제공하는 역할을 수행한다.

백엔드는 REST API 기반으로 설계하였으며 FastAPI 프레임워크를 사용하여 구현하였다. 모든 기능은 API 중심 구조로 개발하여 프론트엔드와 독립적으로 동작할 수 있도록 구성하였다.

전체 흐름은 다음과 같다.

회원가입

↓

사용자 정보 저장

↓

키스트로크 프로파일 저장

↓

로그인 요청

↓

계정 검증

↓

인증 엔진 수행

↓

결과 반환

↓

로그 저장

5.2.2 기술 스택

FastAPI

백엔드 프레임워크로 FastAPI를 사용하였다.

FastAPI를 선택한 이유는 다음과 같다.

- 높은 처리 성능
- 비동기 API 지원
- 자동 Swagger 문서 생성
- Pydantic 기반 데이터 검증
- REST API 개발 용이성

주요 기능

- 회원가입 API
- 로그인 API
- MFA 인증 API
- 사용자 인증 API
- 데이터 조회 API

PostgreSQL

데이터베이스는 PostgreSQL을 사용하였다.

인증 시스템 특성상 사용자 데이터의 무결성과 안정성이 중요하기 때문에 관계형 데이터베이스를 채택하였다.

PostgreSQL을 활용하여 다음 데이터를 저장하였다.

- 사용자 계정 정보

- 키스트로크 프로파일
- 로그인 이력
- 인증 로그
- RBA 학습 데이터

SQLAlchemy ORM

SQLAlchemy ORM을 사용하여 데이터베이스를 관리하였다.

ORM을 활용함으로써

- 객체 중심 개발 가능
- SQL 의존성 감소
- 유지보수성 향상
- 모델 관리 단순화

등의 장점을 확보하였다.

Redis

Redis는 MFA 인증 과정에서 임시 데이터를 저장하기 위해 사용하였다.

Redis를 활용한 이유는 다음과 같다.

- 메모리 기반 고속 처리
- TTL(Time To Live) 지원
- 임시 인증 상태 저장 가능

사용 예시는 다음과 같다.

- MFA 인증 대기 상태
- QR 인증 완료 여부
- 인증 만료 시간 관리

Docker

서비스 배포를 위해 Docker를 사용하였다.

Docker를 통해 개발 환경과 운영 환경의 차이를 최소화하였으며 컨테이너 단위로 서비

스를 관리할 수 있도록 구성하였다.

컨테이너 구성

- FastAPI Container
- PostgreSQL Container
- Redis Container

AWS EC2

최종 서비스는 AWS EC2 인스턴스에 배포하였다.

이를 통해 실제 서버 환경에서 서비스가 동작하도록 구현하였으며 외부 네트워크를 통한 접속 및 테스트가 가능하도록 구성하였다.

5.2.3 데이터베이스 설계

본 시스템은 총 네 개의 핵심 테이블을 중심으로 설계하였다.

Users 테이블

사용자의 기본 계정 정보를 저장한다.

주요 컬럼

컬럼명	설명
id	사용자 고유 번호
username	사용자 계정
hashed_password	암호화된 비밀번호
created_at	가입 시각
last_ip	마지막 로그인 IP
last_device	마지막 로그인 장치

설계 목적 : 사용자의 기본 인증 정보를 관리하고 로그인 시 계정 검증에 활용하기 위함이다.

User_Keystroke_Profiles 테이블

회원가입 시 등록된 키스트로크 프로필을 저장한다.

주요 컬럼

컬럼명	설명
id	프로필 ID
user_id	사용자 ID
raw_profile_data	키스트로크 프로필 데이터
created_at	생성 시각

설계 목적 : 회원가입 시 수집한 15회의 타건 데이터를 저장하여 사용자의 기준 프로필로 활용하기 위함이다.

Keystroke_Logs 테이블

로그인 성공 시 생성되는 키스트로크 이력을 저장한다.

주요 컬럼

컬럼명	설명
id	로그 ID
risk_log_id	인증 로그 참조 ID
keystroke_timing	로그인 시 타건 데이터
created_at	생성 시각

설계 목적 : 사용자의 최신 타건 패턴을 누적 저장하여 프로필 관리 및 인증 기록 분석에 활용하기 위함이다.

RBA_Ready_To_Train 테이블

로그인 시 수집되는 환경 정보를 저장한다.

주요 컬럼

컬럼명	설명
login_timestamp	로그인 시각

컬럼명	설명
user_id	사용자 ID
country	국가
region	지역
city	도시
asn	통신사 정보
browser_name_version	브라우저 정보
os_name_version	운영체제 정보
device_type	장치 종류
resolution	해상도
language	브라우저 언어
rtt	네트워크 RTT
login_successful	로그인 성공 여부

설계 목적 : 사용자의 접속 환경을 저장하고 인증 이력을 관리하기 위함이다.

5.2.4 회원가입 API 구현

회원가입 과정에서는 일반적인 계정 생성 기능 외에도 키스트로크 프로파일 등록 기능을 함께 수행한다.

사용자가 회원가입을 진행하면 다음 데이터가 백엔드로 전달된다.

- 사용자 계정
- 비밀번호
- 키스트로크 이벤트 데이터
- 환경 정보

백엔드는 입력 데이터를 검증한 후 다음 작업을 수행한다.

1. 비밀번호 해시화
2. 사용자 계정 생성
3. 키스트로크 프로파일 저장
4. 사용자 환경 정보 저장

비밀번호는 평문으로 저장하지 않고 해시값만 저장하도록 구현하였다.

5.2.5 로그인 API 구현

로그인 API는 시스템의 핵심 진입점 역할을 수행한다.

로그인 요청이 발생하면 다음 순서로 처리된다.

1단계 : 사용자 계정 조회

username 조회

↓

사용자 존재 여부 확인

2단계 : 비밀번호 검증

입력 비밀번호

↓

해시 비교

↓

일치 여부 확인

3단계 : 인증 엔진 호출

키스트로크 인증

+

환경 분석

4단계 : 정책 엔진 결과 수신

ALLOWED

MFA_REQUIRED

DENIED

5단계 : 인증 결과 저장

인증 성공 시

- 로그인 이력 저장
- 환경 정보 저장

을 수행하도록 구현하였다.

5.2.6 MFA 인증 구현

중위험 사용자의 경우 추가 인증을 수행하도록 설계하였다.

로그인 과정에서 추가 인증이 필요한 경우

MFA_REQUIRED

상태를 반환한다.

사용자는 모바일 인증 페이지에서 인증을 수행하며 인증 성공 시 Redis에 저장된 인증 상태를 갱신한다.

백엔드는 주기적으로 해당 상태를 확인하여 인증 완료 여부를 검증한다.

5.2.7 예외 처리 및 보안 설계

시스템 안정성을 확보하기 위하여 다양한 예외 처리 로직을 구현하였다.

입력 데이터 검증

Pydantic 모델을 사용하여 모든 요청 데이터를 검증하였다.

검증 항목

- 필수 값 존재 여부
- 데이터 타입 검증
- 요청 형식 검증

비밀번호 보호

사용자의 비밀번호는 해시화하여 저장하였다.

이를 통해 데이터베이스가 노출되더라도 실제 비밀번호가 유출되지 않도록 설계하였다.

API 예외 처리

모든 주요 API에 대해 예외 처리 로직을 구현하였다.

예시

- 존재하지 않는 사용자
- 잘못된 비밀번호
- 비정상 요청 데이터
- 만료된 MFA 인증

인증 로그 관리

로그인 성공 및 실패 기록을 저장하여 추후 분석이 가능하도록 설계하였다.

이를 통해 비정상 접근 시도 추적 및 보안 감사가 가능하도록 구현하였다.

6. 결론 및 프로젝트 의의

6.1 인증 방식 비교 및 지속성

비밀번호 방식은 생체 인증에 비해 오인식률과 오거부율이 모두 높지만, 구축 비용이 저렴하고 사용자에게 친숙하다는 장점이 있다. 본 연구는 이러한 비밀번호 체계를 유지하면서도 보안성을 획기적으로 높이는 방안에 집중한다.

6.2 최종 결론

이번 프로젝트는 구조는 MFA이되 경험은 SFA인 인증 시스템을 구현했다는 데 의의가 있다. 평상시에는 비밀번호만으로 간편하게 로그인하되, 이상 징후 발생 시에만 배후의 키스트로크와 RBA가 작동하여 2차 인증을 요구함으로써 보안과 사용자 경험을 동시에 만족시키는 모델을 제시한다.

1. 참고 문헌

국내 학술 논문

- 신지안, 김휘강. (2020). "키스트로크 다이내믹스 및 적응형 위험 기반 인증방식을 활용한 사용자 인증 시스템 구현." *대한전기학회 논문지*, 69(4), 267-274.
- 김경훈, 권태경. (2017). "모바일 핀테크 서비스에서 이용 가능한 인증 수단의 사용

성, 안전성 분석 연구." *정보보호학회/논문지*, 27(4), 843-853.

해외 논문 및 벤치마크 데이터셋

- Killourhy, K. S., & Maxion, R. A. (2009). "Comparing Anomaly-Detection Algorithms for Keystroke Dynamics." *In Proceedings of the IEEE International Conference on Dependable Systems and Networks (DSN-2009)*. (Keystroke Dynamics - Benchmark Data Set).
- Wiefling, S., et al. (2021). "Login Data Set for Risk-Based Authentication (RBA)." *Synthesized login feature data from a large-scale online single sign-on (SSO) service in Norway (2020-2021)*.
- Sheetal. (2025). "A Data-Driven Comparative Analysis of Password and Biometric Authentication Systems." *International Journal on Science and Technology (IJSAT)*, 16(4), 1-13.